# Search-Based Software Engineering

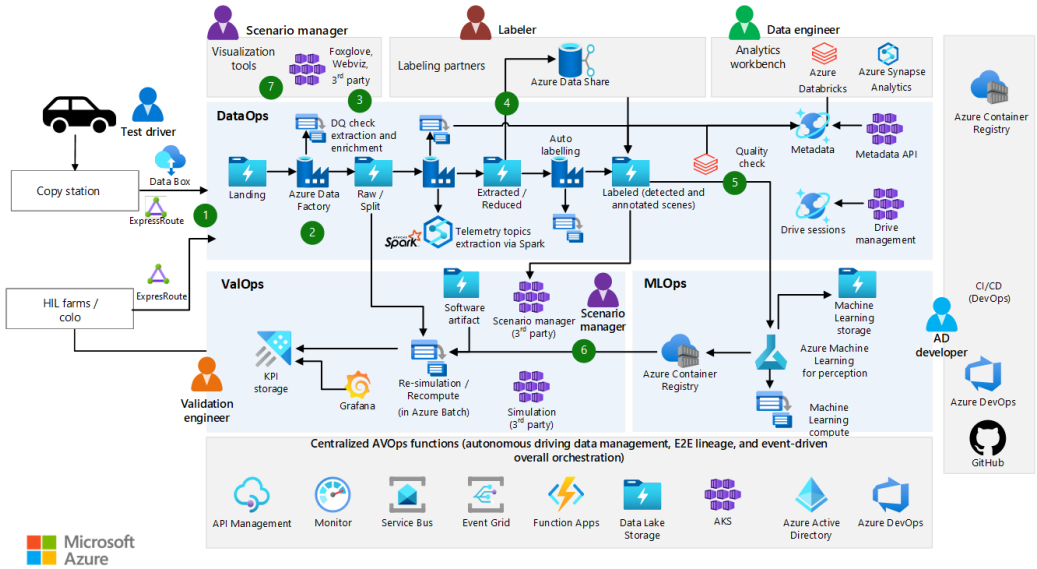
Sol Zilberman

Michigan State University - Computer Science & Engineering

FS25 CSE 435

# Modern software challenges

- Systems are becoming **more complex**
- Software engineers choose b/w many possible **alternatives**



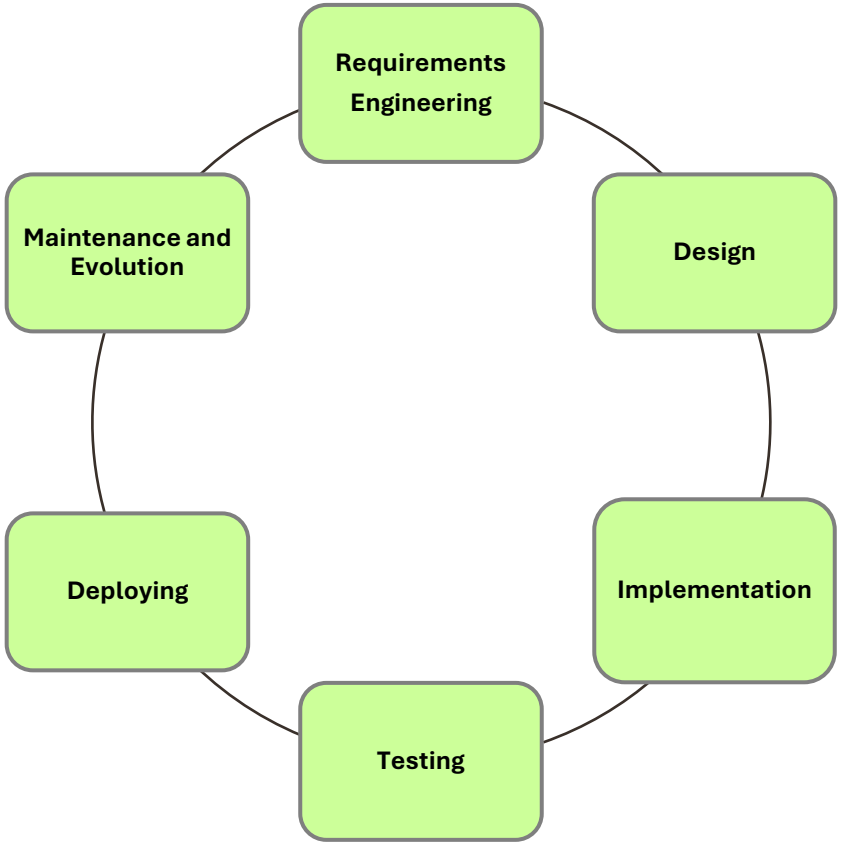

https://learn.microsoft.com/en-us/azure/architecture/solution-ideas/articles/avops-architecture

https://blog.bigml.com/wp-content/uploads/2013/04/decision_tree.png?w=640

# Decisions problems in software engineering

What is the set of requirements that balances cost and safety?

What is the best sequence of refactoring steps to apply to this system?




What is the best architecture configuration to enhance maintainability?

What hosting configuration best balances cost and speed?

What is the best allocation of resources to this software development project?

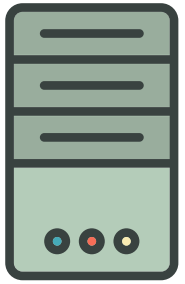
What is the smallest set of test cases that covers all branches in this program?

# Finding better solutions

- Many SWE problems can be framed as **search** problems
- Find the **“best”** solution for some **criteria**

Cost

Resources

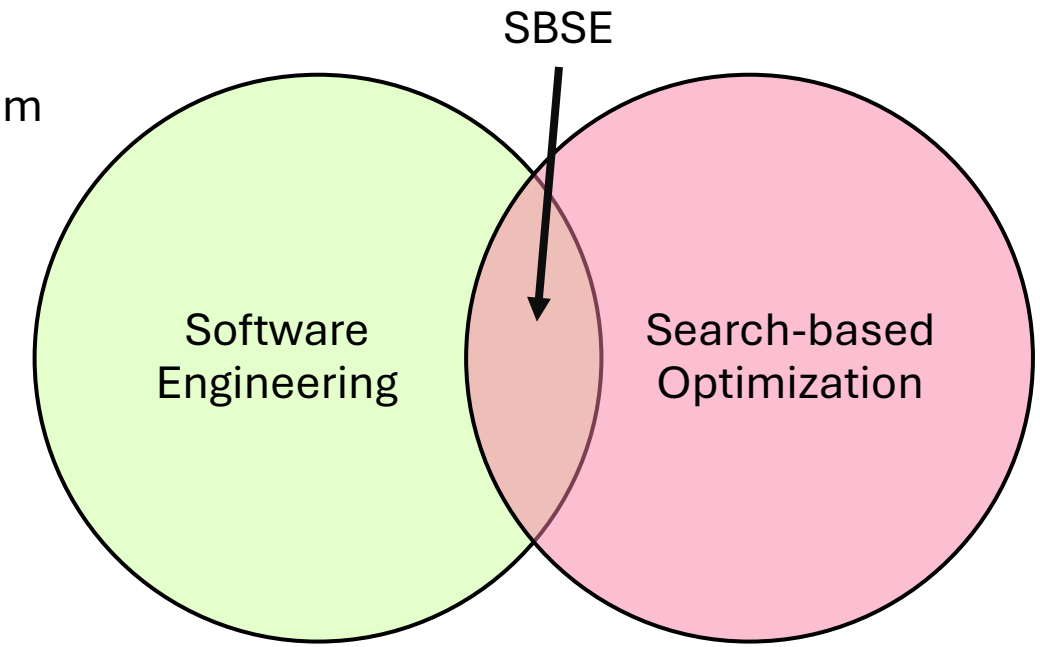


Time

# Search-based Software Engineering

- Apply **search techniques** to software engineering problems, guided by an **evaluation function** that captures properties of the acceptable software artifacts we seek [1]

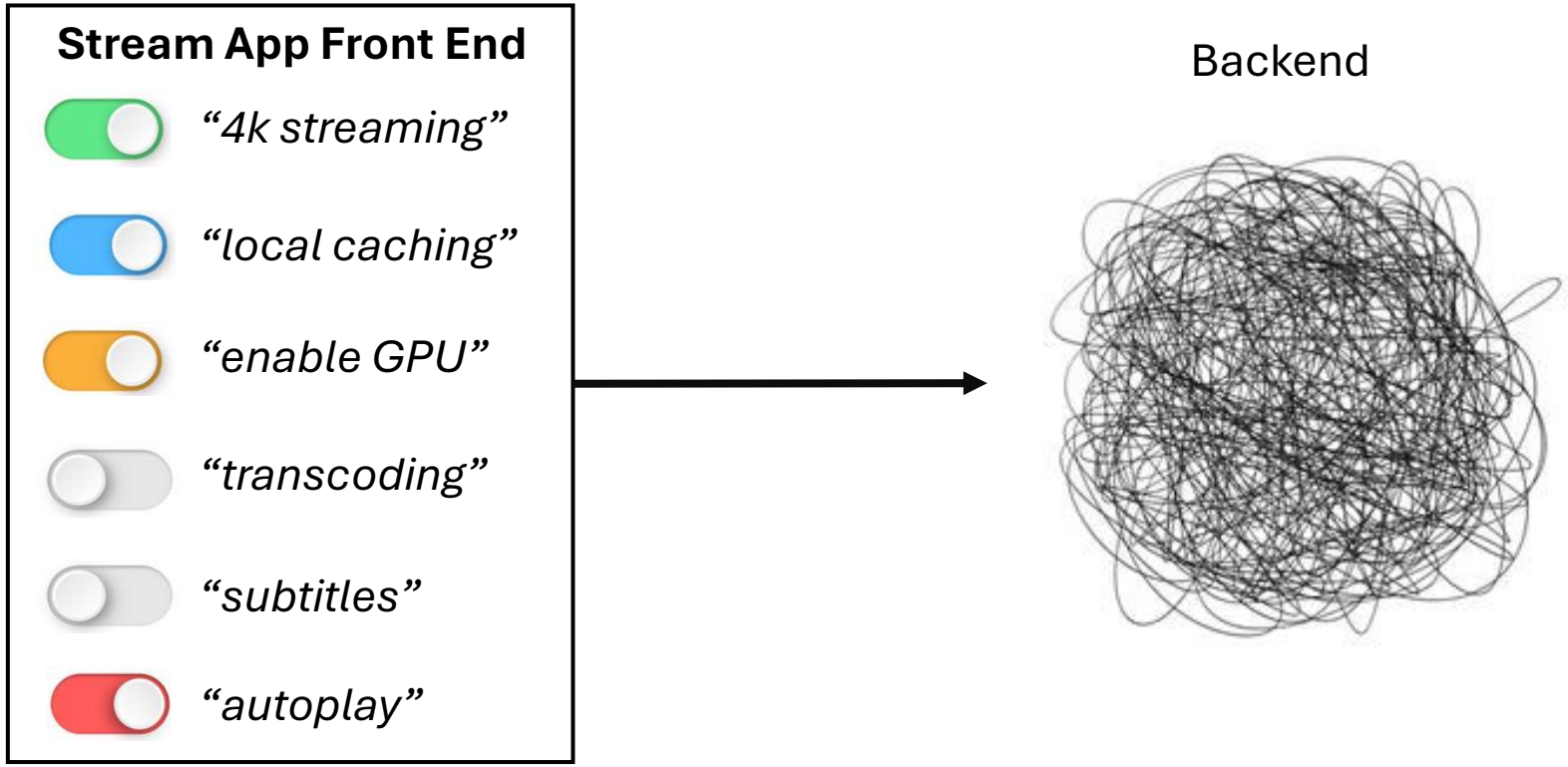
1. How to **frame** SWE problem as **search problem**?




2. How to **intelligently search** for the solution?

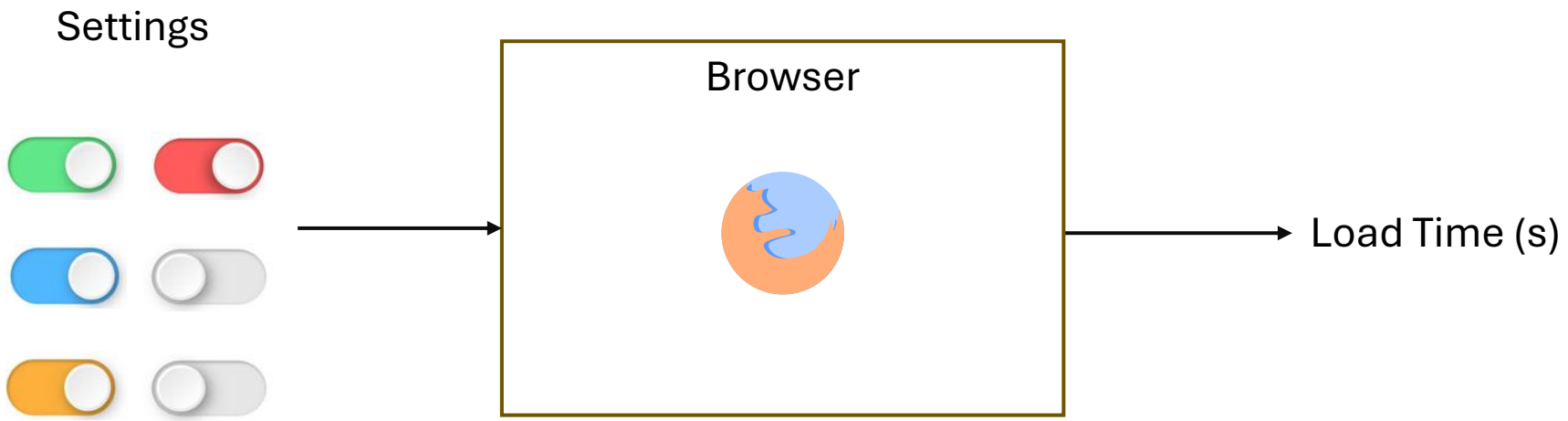
# Motivating software testing example

- Your app provides a number of **configuration options** to the user.
- **Requirement**: For any configuration, page load occurs < 5s



# Motivating software testing example

- Find a test case (combination of settings) that **maximizes** page load time

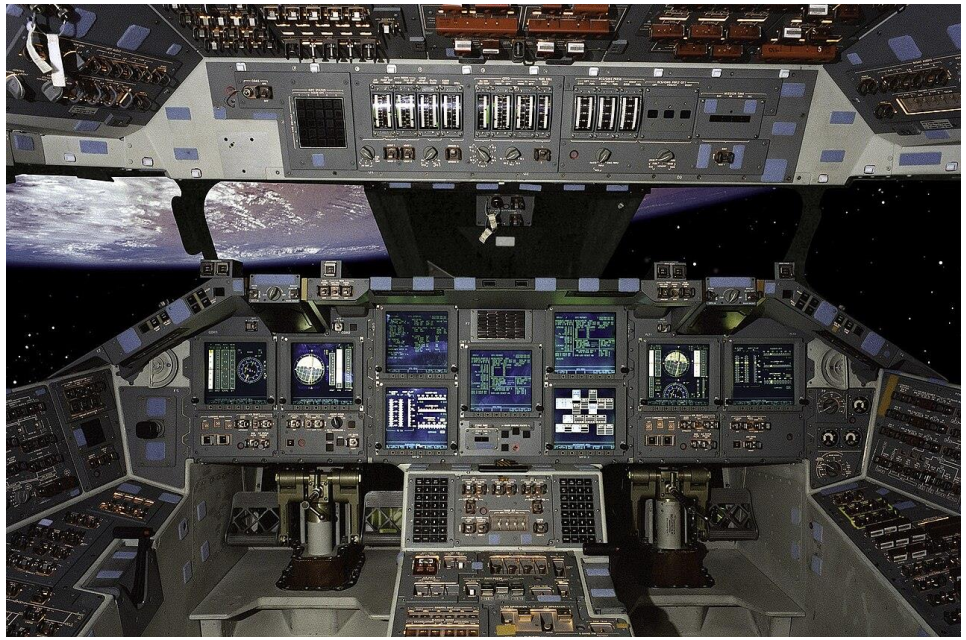



Only need to test $2^6$=64 combinations

# A more challenging problem

- What if we have 1000+ settings, each with multiple options?

https://en.wikipedia.org/wiki/SEPECAT_Jaguar#/media/File:Cockpit_of_Jaguar_GR.3A.jpg

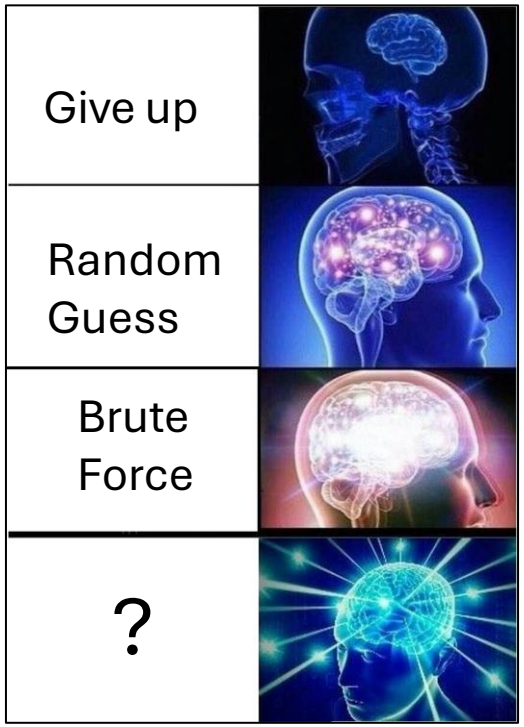


https://en.wikipedia.org/wiki/Space_Shuttle#/media/File:STSCPanel.jpg

$3^{1000} \approx 2^{1584}$ possible settings! Our universe only has $\approx 2^{500}$ atoms

# SBSE: Recap

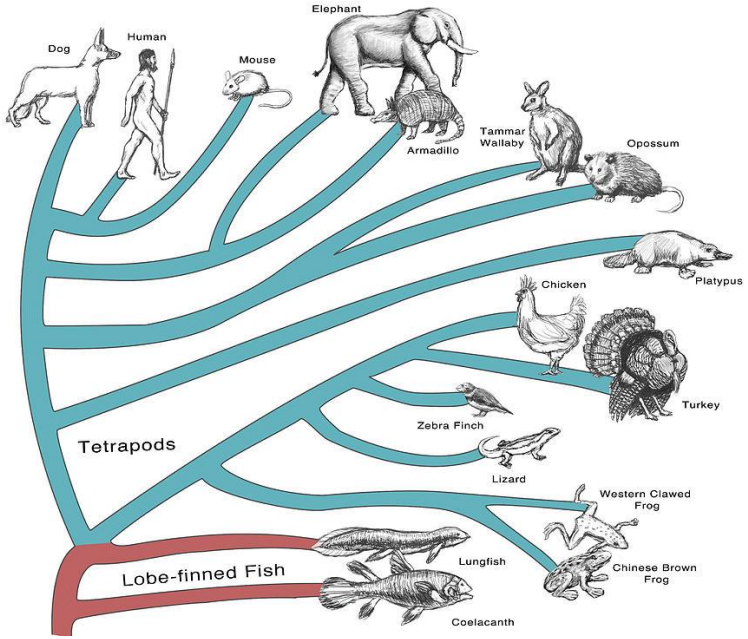
1. View SWE problems as a **search problem** ✅
2. Now, how do we **intelligently** solve the SWE search problems ❓



# SBSE: Evolutionary Approach

# Evolutionary Computation (EC)

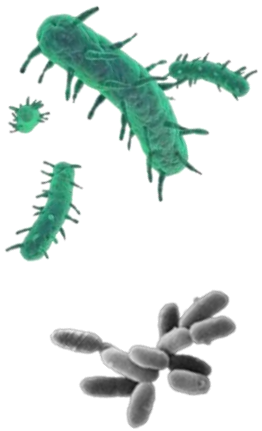
- Search/optimization technique inspired by **Darwinian Evolution**
- Applications
  - Software testing
  - Robotics
  - Design problems
  - Scheduling
  - etc.




https://images.fineartamerica.com/images-medium-large-5/1-evolutionary-tree-animals-spencer-sutton.jpg

# Why do we expect EC to work?

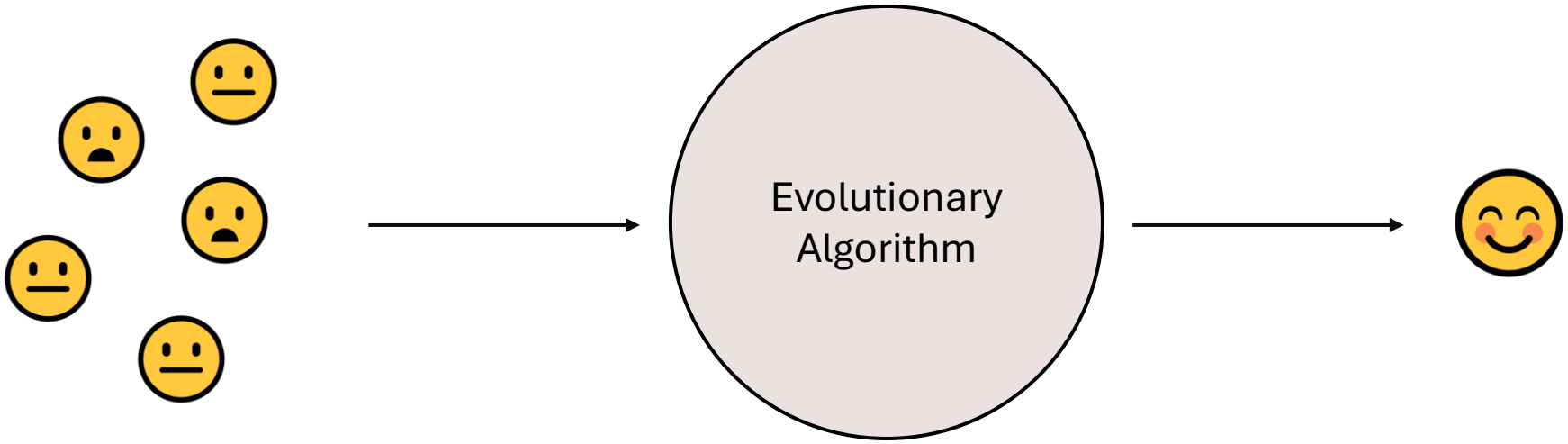
- Nature finds **creative** solutions to complex problems
- From **simple** cells to **complex** life forms



Natural
Evolution

# Key Idea of EC

- Initialize set of random solutions
- Evolve them to solve problem
- Keep best solution after N generations



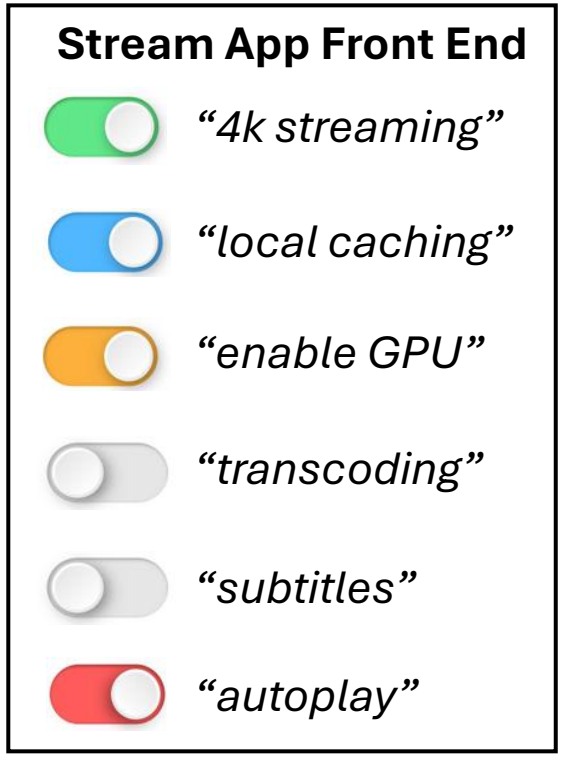
# Evolutionary Computation “Ingredients”

- Representation
- Fitness Function
- Selection
- Crossover
- Mutation

```python
def search():
    # create random population
    pop=create_population()

    for _ in range(NUM_GENERATIONS):
        # evaluate and select best ones
        pop=select(pop, fit_func)
        # combine to produce next gen
        next_gen=crossover(pop)
        # apply random mutations
        mutated=mutate(next_gen)
        pop=mutated # update population

    # sort and return best
    pop = sorted(pop)
    return pop[-1]
```

# EC - Representation

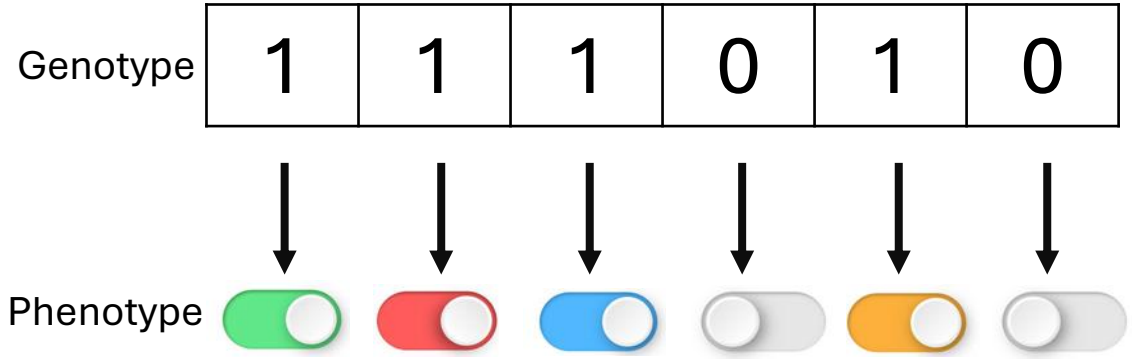
- **Encodes** a **candidate** solution in computer-friendly format
- How can we **represent** a set of on/off toggles?



# EC - Representation

- **Encodes** a **candidate** solution in computer-friendly format
- Use binary string

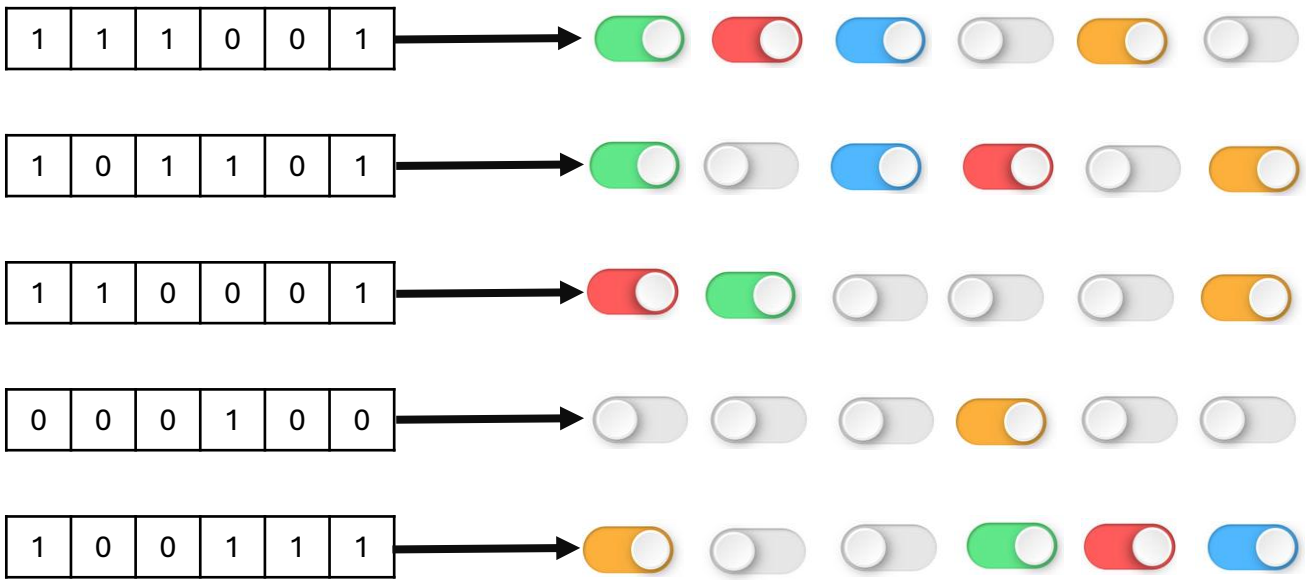
```python
GENES = [1, 0]
NUM_TOGGLES = 6

def random_genome():
    return [random.choice(GENES) for _ in range(NUM_TOGGLES)]

class Individual:
    def __init__(self):
        self.genome = random_genome()

random_solution = Individual()
```



# EC - Population

- A group of candidate **solutions**
- Contains all the “knowledge” learned so far

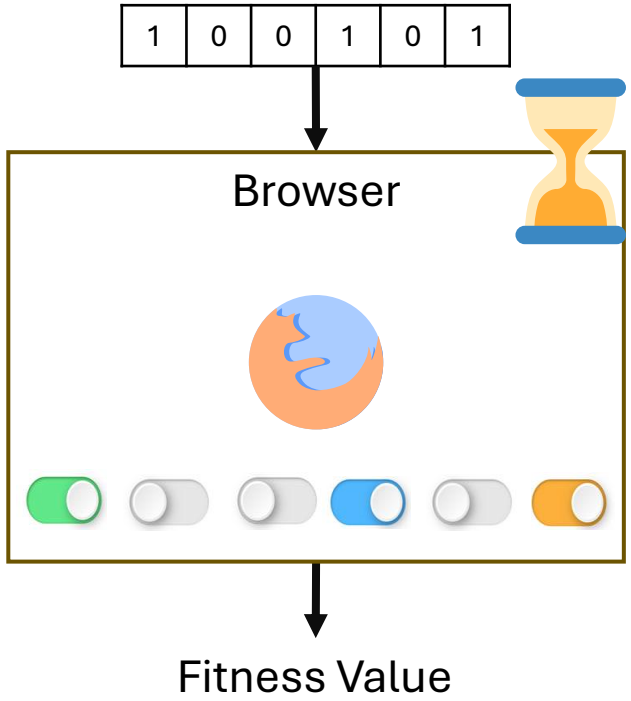
```python
POPULATION_SIZE = 100
population = []
for i in range(POPULATION_SIZE):
    population += [Individual()]
```



# EC – Fitness Function

- **Evaluates** how “good” a solution is
- Captures **requirements** to optimize

```python
def evaluate(individual):
    app = Application(individual.genotype)
    start_time = time.time()
    app.load()
    end_time = time.time()
    return end_time - start_time
```

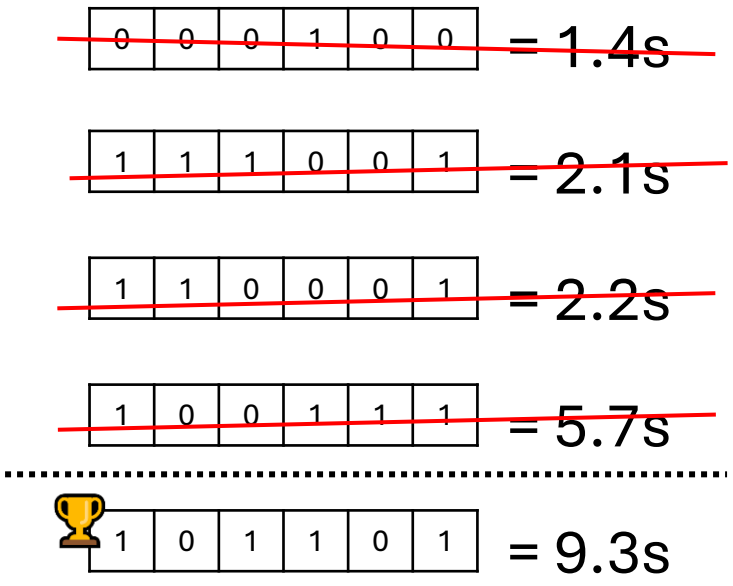


# EC - Selection

- **Keeps** only the “best” solutions from the population

```python
def selection(population, tournament_size):
    survivors = []

    for _ in range(POPULATION_SIZE):
        # get players
        tournament = random.sample(population, tournament_size)
        # evaluate and sort
        tournament.sort(key=lambda ind: evaluate(ind))
        survivors.append(tournament[0])

    return survivors
```

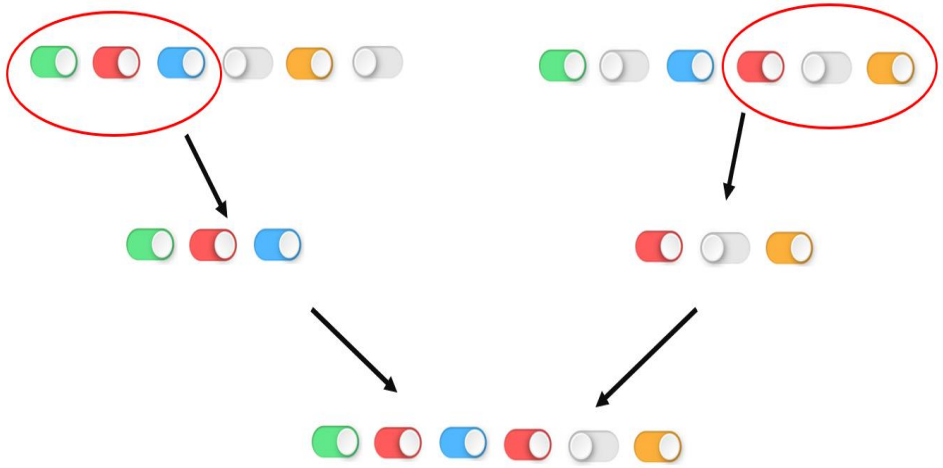
Tournament



# EC - Crossover

- Randomly **combines** two solutions to create an “offspring”

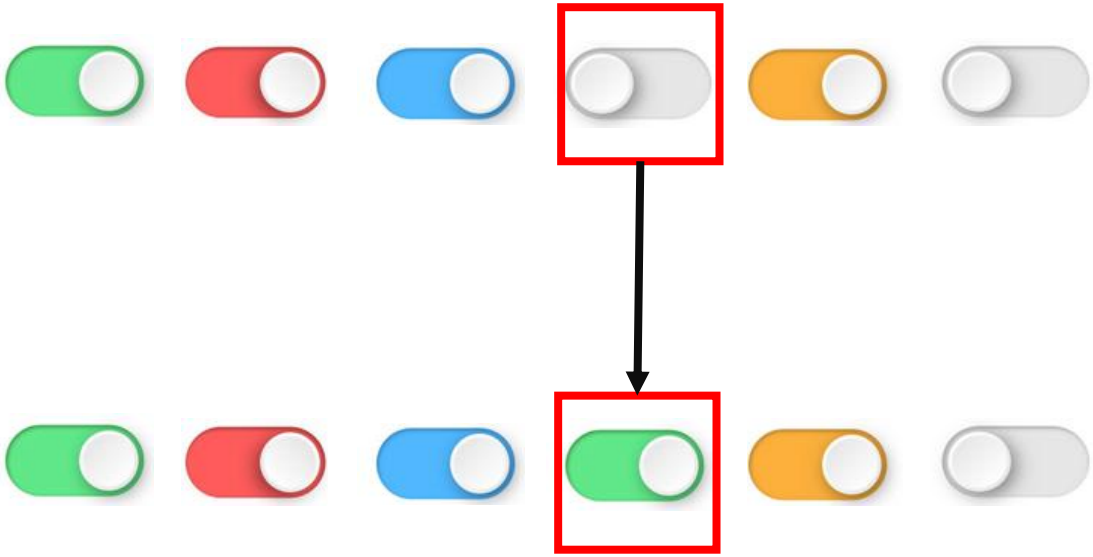
```python
def crossover(parent1, parent2):
    # pick xover point
    crossover_point = random.randint(1, len(parent1.genome) - 1)

    # create new genomes by combining parents
    child1_genome = parent1.genome[:crossover_point] + \
                    parent2.genome[crossover_point:]
    child2_genome = parent2.genome[:crossover_point] + \
                    parent1.genome[crossover_point:]

    # generate children
    child1 = Individual()
    child1.genome = child1_genome
    child2 = Individual()
    child2.genome = child2_genome

    return child1, child2
```



# EC - Mutation

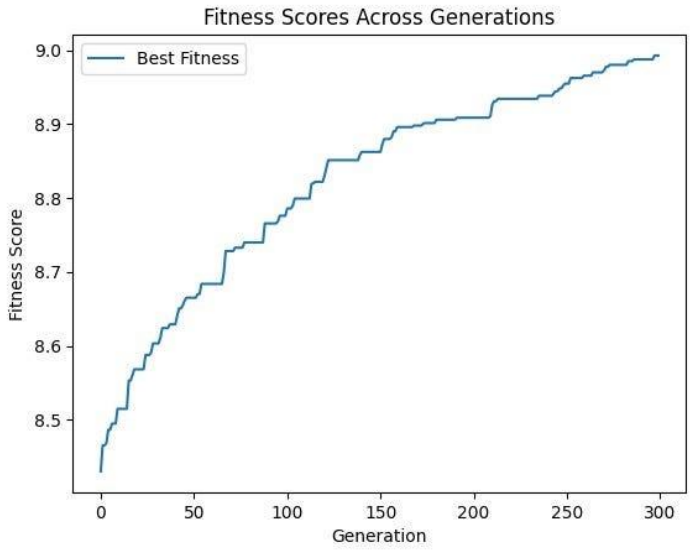
- **Randomly** modifies “offspring” with some probability *p*

```python
def mutate(individual, mutation_rate=0.1):
    for i in range(NUM_TOGGLES):
        # mutate gene with probability of mutation_rate
        if random.random() < mutation_rate:
            individual.genome[i] = random.choice(GENES)
    return individual
```



# EC – Putting it all together

- As population evolves, fitness improves!

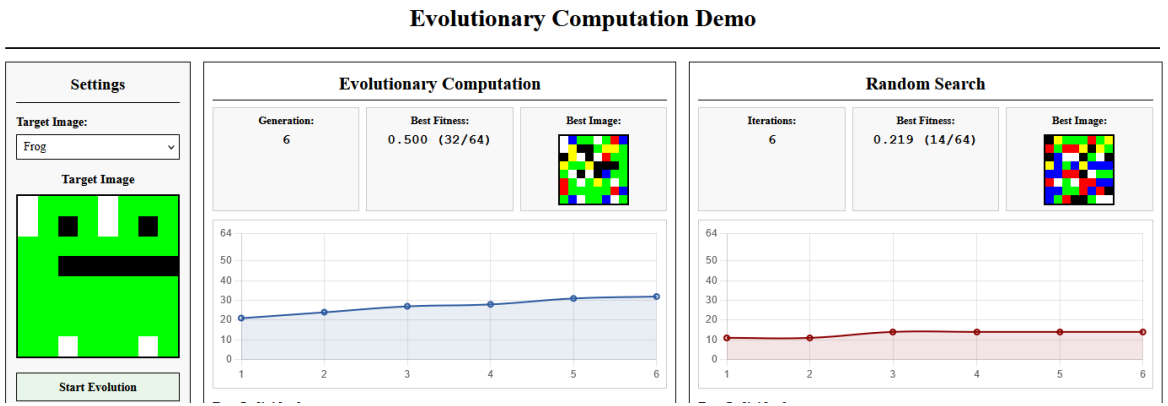



```python
def run_search():
    population = [Individual() for _ in range(POPULATION_SIZE)]

    # run n generations of evolution
    for generation in range(NUM_GENERATIONS):
        # selection
        population = selection(population, TOURNAMENT_SIZE)

        # create next generation
        next_generation = []
        # apply EC operators
        while len(next_generation) < POPULATION_SIZE:
            # xover
            parent1, parent2 = random.sample(population, 2)
            child1, child2 = crossover(parent1, parent2)
            # mutation
            mutate(child1, MUTATION_RATE)
            mutate(child2, MUTATION_RATE)

            next_generation.extend([child1,child2])

        population = next_generation
    # final evaluation
    population.sort(key=lambda ind: evaluate(ind))
    return population[0]
```

# Demo

- https://solzilberman.github.io/pixel-art-evolver/

```
export const GENES = [
    '#FFFFFF',
    '#000000',
    '#FF0000',
    '#00FF00',
    '#0000FF',
    '#FFFF00',
];

export const TARGET_SOLUTION = {
    frog: [
        [0, 3, 3, 3, 0, 3, 3, 3],
        [0, 3, 1, 3, 0, 3, 1, 3],
        [3, 3, 3, 3, 3, 3, 3, 3],
        [3, 3, 1, 1, 1, 1, 1, 1],
        [3, 3, 3, 3, 3, 3, 3, 3],
        [3, 3, 3, 3, 3, 3, 3, 3],
        [3, 3, 3, 3, 3, 3, 3, 3],
        [3, 3, 0, 3, 3, 3, 0, 3]
    ],
}
```

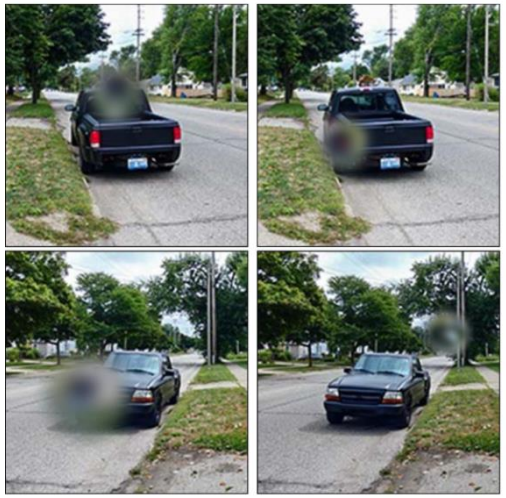



$6^{64} \approx 2^{165}$ possible solutions (*> number of atoms on Earth*)

# Applications

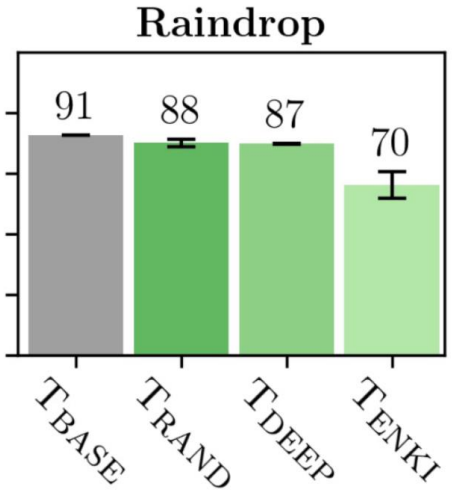
# Applications – AV Robustness

- Find raindrops that lead to most diverse AV sensor outputs
- Representation?



(c) Simulated Droplets

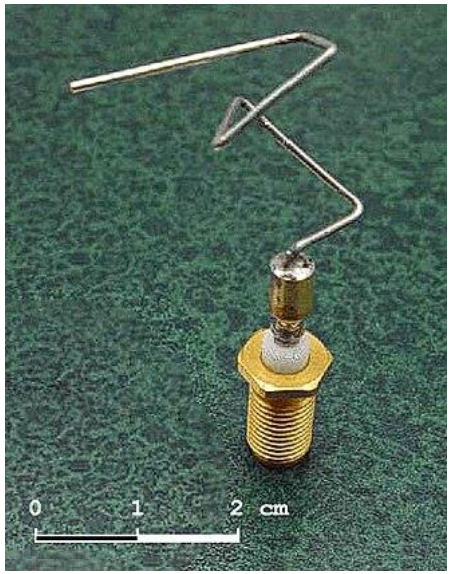
[raindrop_x, raindrop_y, raindrop_radius, raindrop_blur]




Langford, Michael Austin, and Betty HC Cheng. "Enki: A diversity-driven approach to test and train robust learning-enabled systems." *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* 15.2 (2021): 1-32.

# Applications – NASA Antenna Design

- Evolve antenna to maximize **coverage**
- Representation?



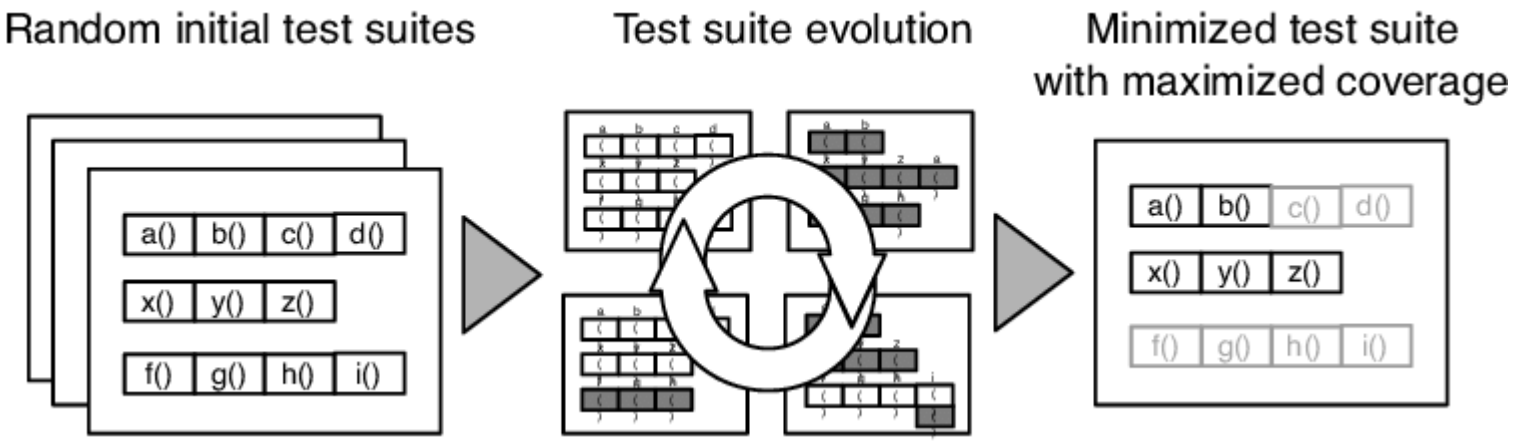

$$[x_{\text{start}}, y_{\text{start}}, z_{\text{start}}, x_{\text{end}}, y_{\text{end}}, z_{\text{end}}, \theta_{\text{angle}}, \ldots]$$

https://en.wikipedia.org/wiki/Evolved_antenna

# Applications – Software Testing

- Generate unit tests that **maximize** code coverage

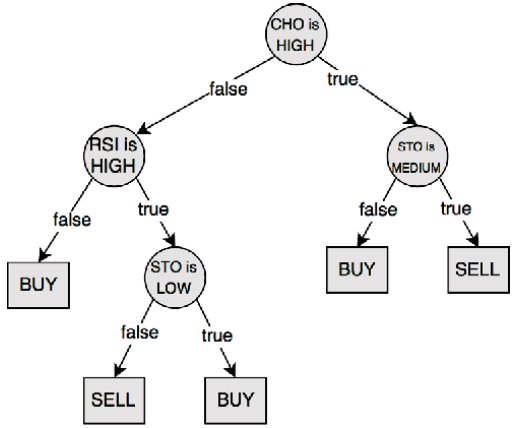
EVOSUITE
Automatic Test Suite Generation for Java




https://www.researchgate.net/publication/224256027/figure/fig1/AS:669035532410885@1536521909266/The-EVOSUITE-process-A-set-of-randomly-generated-test-suites-is-evolved-to-maximize.png

# Applications – Algorithmic Trading

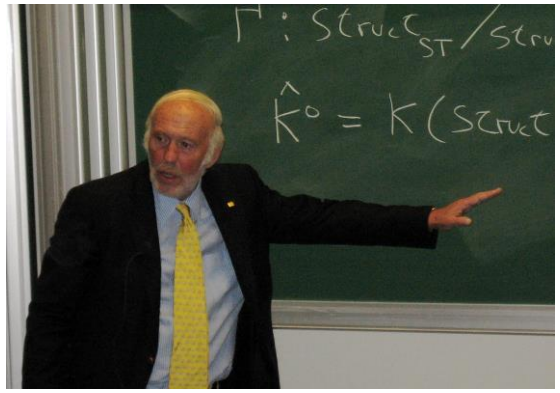
- Used by high-profile trading firms to optimize investment **strategies**




https://d3i71xaburhd42.cloudfront.net/6949f3acae3ff3a229a89c86b5a30b5f3e4830ae/2-Figure1-1.png

https://miro.medium.com/v2/resize:fit:1400/0*Wk9xHBsFVuWOw8tt.png

Jim Simons



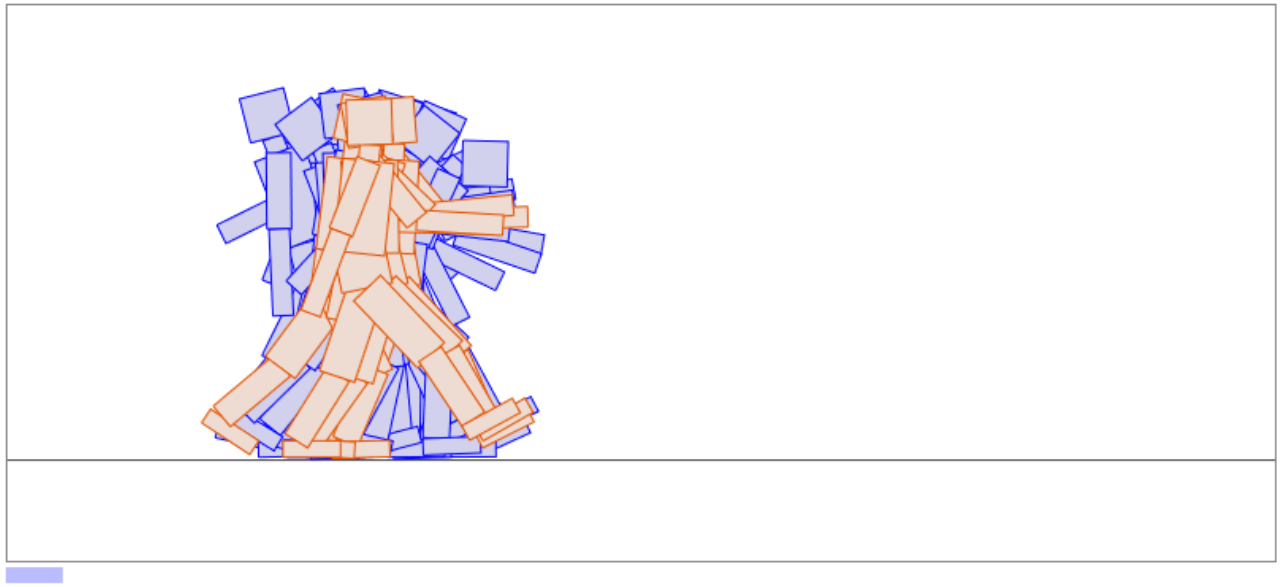
https://en.wikipedia.org/wiki/Jim_Simons

**not financial advice*

# Applications – Robotics

- Bi-pedal robot control
- Representation?

$$[c_1, c_2, \ldots, c_n], c \in \{\mathrm{q},\ \mathrm{w},\ \mathrm{o},\ \mathrm{p}\}$$



https://rednuht.org/genetic_walkers/

# Application – Adaptive Systems

- Evolve virtual creature to win food-seeking competition

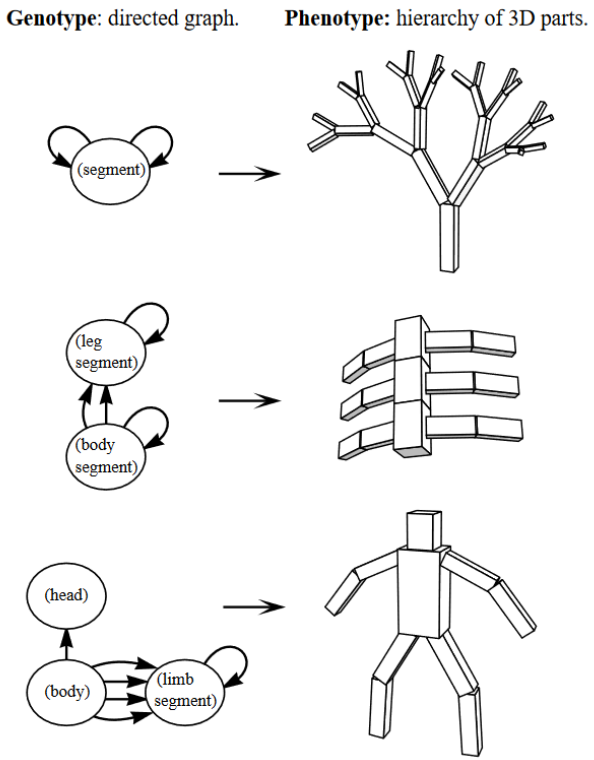


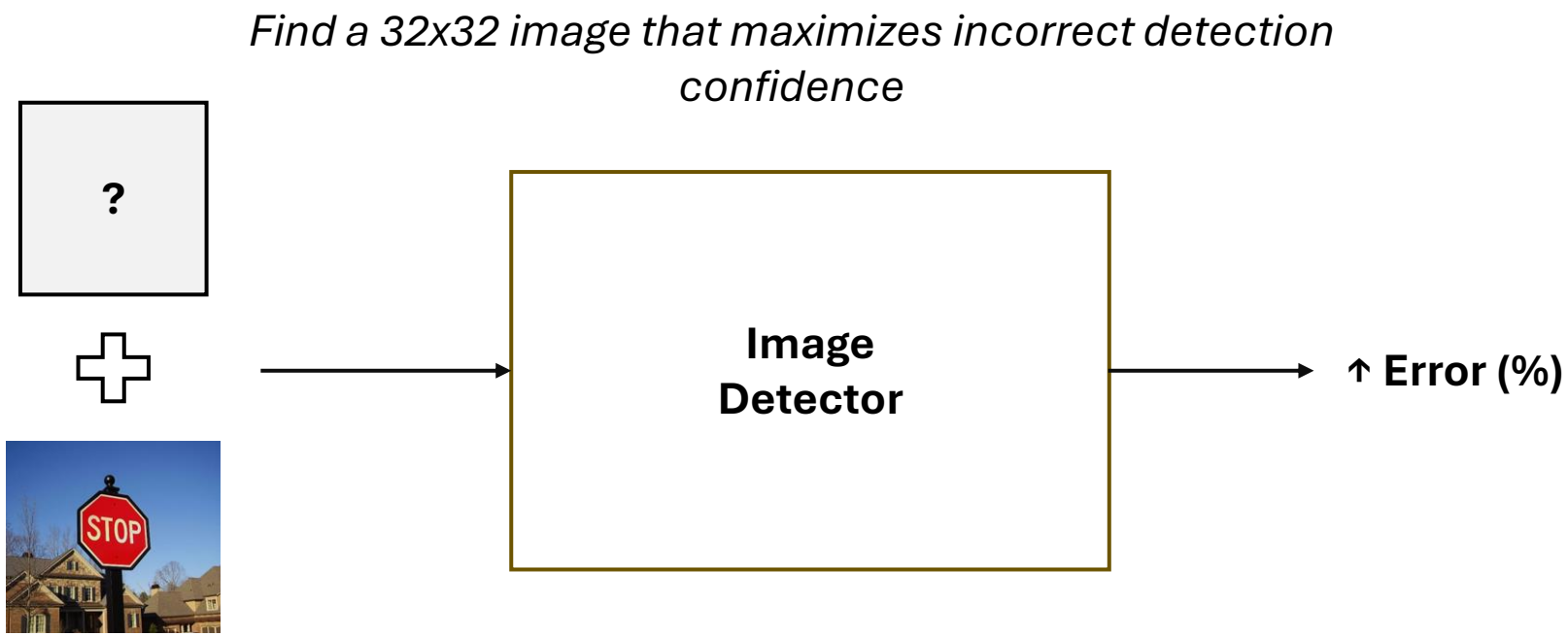

**Figure 1**: Designed examples of genotype graphs and corresponding creature morphologies.

http://www.karlsims.com/papers/siggraph94.pdf

https://www.youtube.com/watch?v=JBgG_VSP7f8

# Applications – Adversarial Attacks

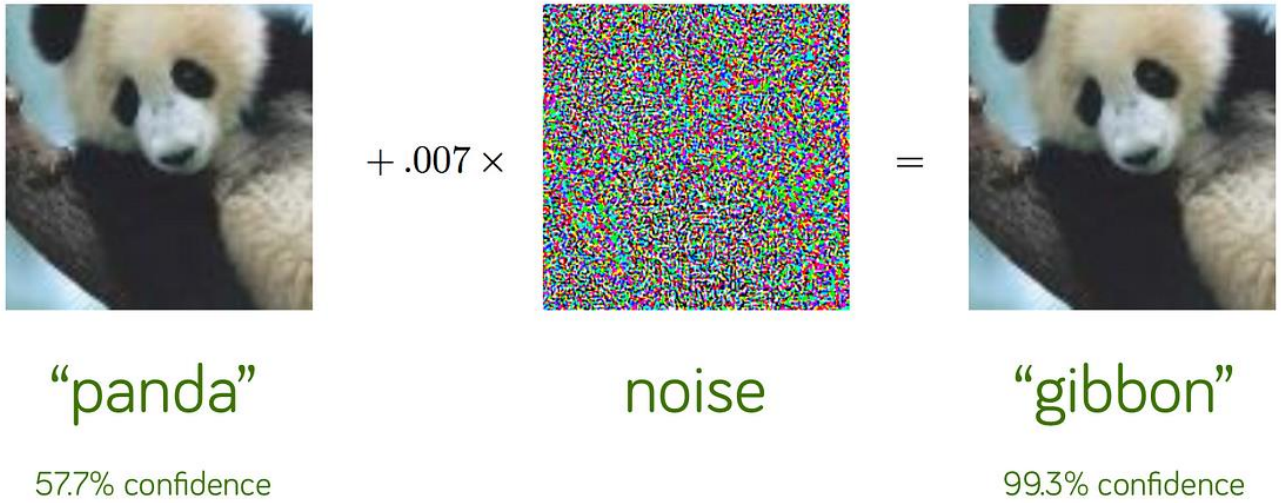
- Find a 32x32 **image mask** that **maximizes** detection error



# Applications – Adversarial Attacks

- How can we represent our input?

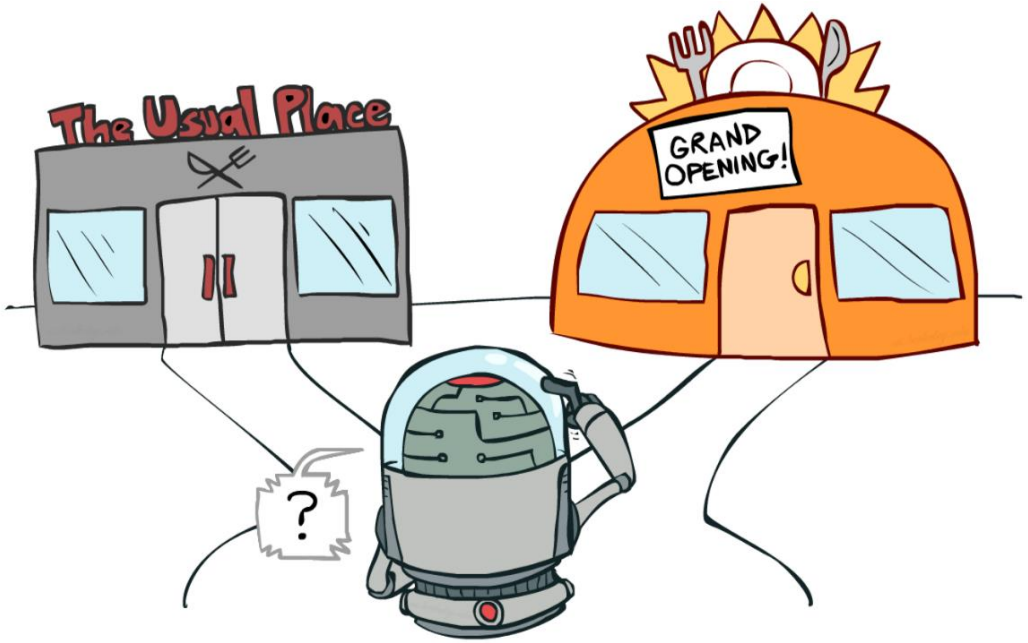
$$\begin{bmatrix} 0.12 & 0.43 & 0.38 & 0.01 & 0.97 & \cdots \\ 0.12 & 0.33 & 0.42 & 0.61 & 0.69 & \cdots \\ 0.56 & 0.03 & 0.76 & 0.67 & 0.70 & \cdots \\ 0.71 & 0.99 & 0.12 & 0.71 & 0.31 & \cdots \\ 0.83 & 0.79 & 0.34 & 0.00 & 0.35 & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \end{bmatrix}$$



# EC Theory

# EC Theory – Crossover vs. Mutation

- Why do we need both?
- exploitation vs. exploration




https://huggingface.co/datasets/huggingface-deep-rl-course/course-images/resolve/main/en/unit1/exp_2.jpg

# EC Theory – “No free lunch”

- There is **no single algorithm** that is best for all problems
- Understanding target problem helps design ECs

**Hyperparameters**
- Population size
- Selection method
- Crossover rate
- Mutation rate
- Crossover operator
- Mutation operator
- Number of generations
- Fitness function
- Elitism rate
- Termination criteria
- Parent selection pressure
- Tournament size (for tournament selection)
- Recombination rate
- Initialization method

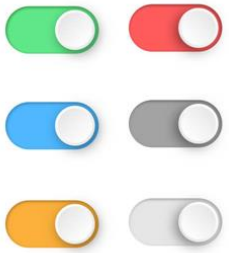
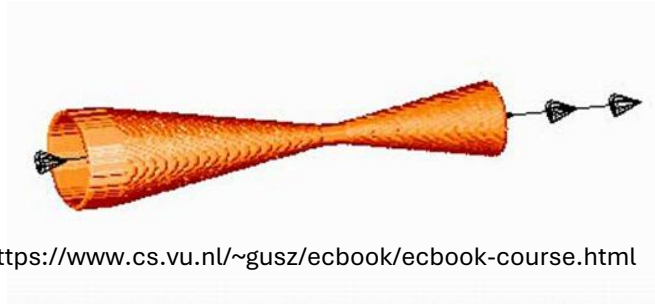
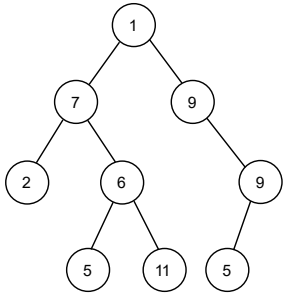
**Crossover Types**
- One-point crossover
- Two-point crossover
- Uniform crossover
- Arithmetic crossover
- Blend crossover (BLX-alpha)
- Simulated binary crossover (SBX)
- Partially matched crossover (PMX)
- Cycle crossover (CX)
- Order crossover (OX)
- Edge recombination crossover
- Heuristic crossover

**Mutation Types**
- Bit-flip mutation
- Swap mutation
- Inversion mutation
- Scramble mutation
- Gaussian mutation
- Uniform mutation
- Non-uniform mutation
- Polynomial mutation
- Boundary mutation
- Random reset mutation

**Selection Types**
- Roulette wheel selection
- Tournament selection
- Rank-based selection
- Truncation selection
- Steady-state selection
- Stochastic universal sampling (SUS)
- Elitism
- Fitness-proportionate selection
- Random selection
- Boltzmann selection

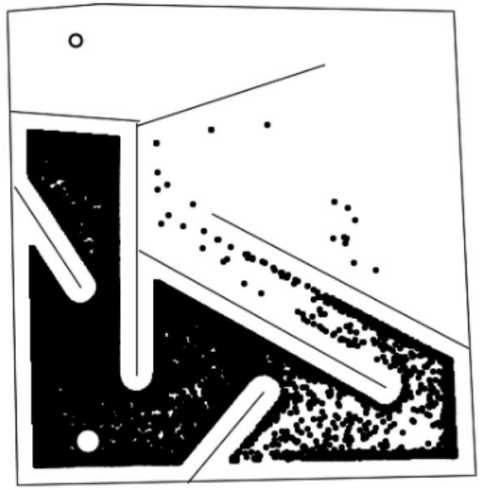
Katoch, Sourabh, Sumit Singh Chauhan, and Vijay Kumar. "A review on genetic algorithm: past, present, and future." *Multimedia tools and applications* 80 (2021): 8091-8126.

# EC Theory – Different Representations

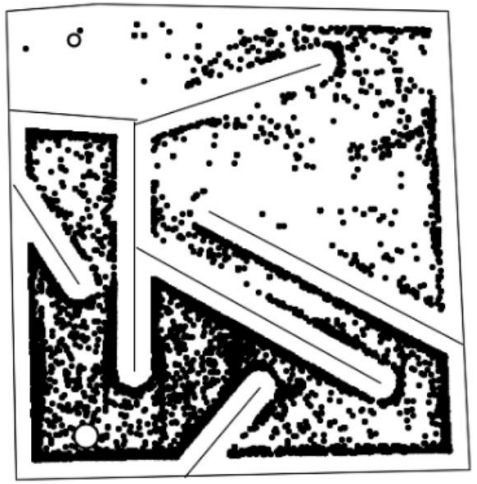
| Genetic Algorithm | Evolutionary Strategy | Genetic Programming |
|---|---|---|
| • Binary<br><br>1 0 0 1 0 1<br><br>  | • Real-valued numbers<br><br>0.12 3.5 2.2 0.84 0.06 1.1<br><br>  <br>https://www.cs.vu.nl/~gusz/ecbook/ecbook-course.html | • Trees<br><br>  <br><br><br>```python<br>def twoNumberSum(array, targetSum):<br>    for i in range(0, len(array)):<br>        for j in range(i+1, len(array)):<br>            if array[i] + array[j] == targetSum:<br>                return ([array[i], array[j]])<br>    return []<br>```<br>(Python 3.8.10) |

# Challenges – Deceptive Landscape

- What if we don’t know a good fitness function?



(d) Hard Map Fitness

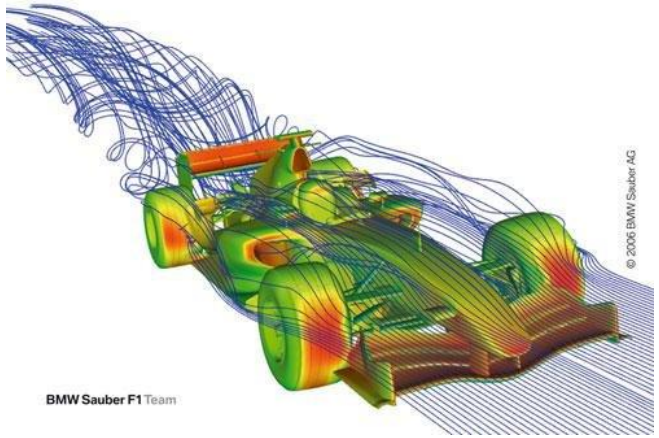


(b) Hard Map Novelty

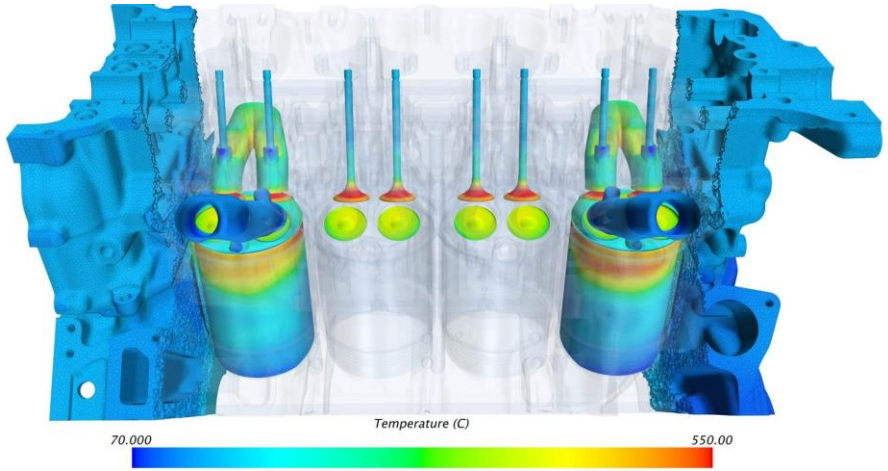
Lehman, Joel, and Kenneth O. Stanley. "Abandoning objectives: Evolution through the search for novelty alone." *Evolutionary computation* 19.2 (2011): 189-223.

# Challenges – Expensive Fitness Calculations

- Some evaluations can take 12+ hours
- E.g., computational fluid dynamics



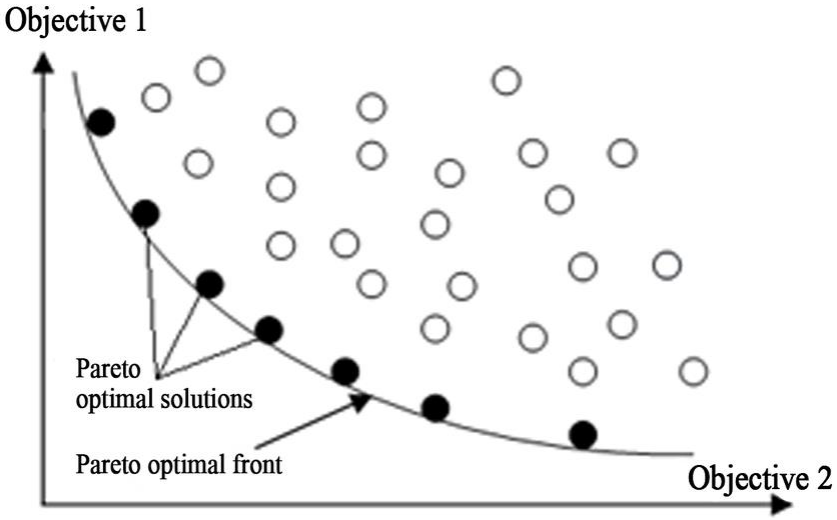

https://www.researchgate.net/publication/268223531/figure/fig9/AS:668575010390020@1536412112352/CFD-Simulation-of-a-Formula-1-car-17.jpg




https://images.squarespace-cdn.com/content/v1/5d7752374ec1dd192cb482d6/9d24bded-87ed-4311-972f-d5413fc88db1/03_analisi_termica_componenti-2048x986.jpeg

# Challenges – Multiple Objectives

- What if we have multiple, **competing** objectives?
- E.g., minimize AV weight *and* maximize AV safety

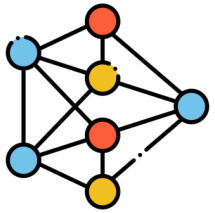



https://www.researchgate.net/publication/301936709/figure/fig18/AS:358806367293454@1462557512846/Non-dominated-solutions-for-two-contradictory-objective-optimization-problems.png

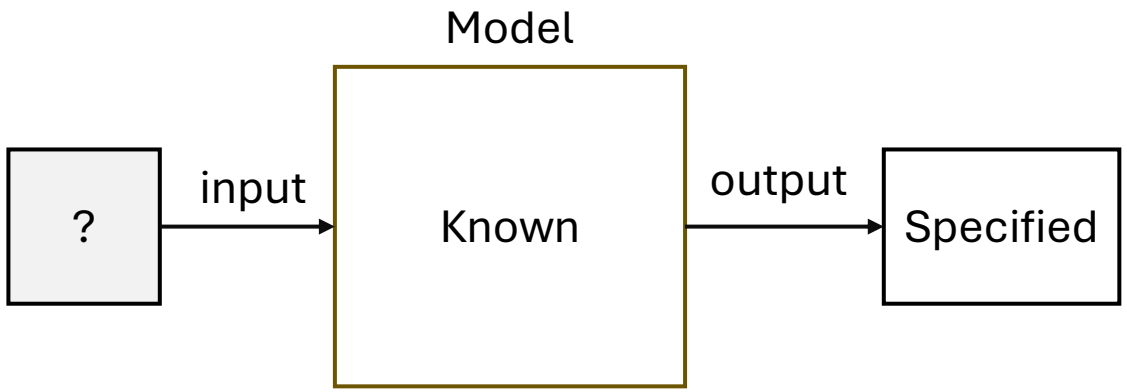
# A broader perspective

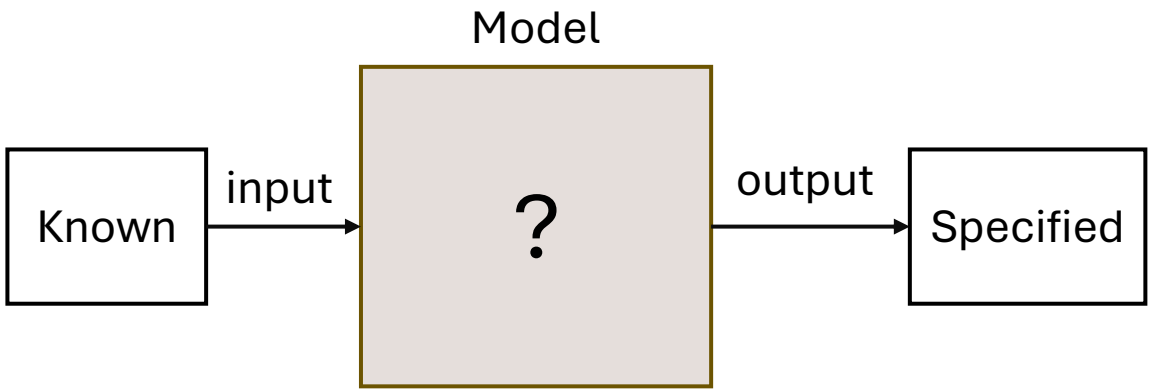
# Evolutionary Computation vs AI

- Have model of problem
- Looking for specific input



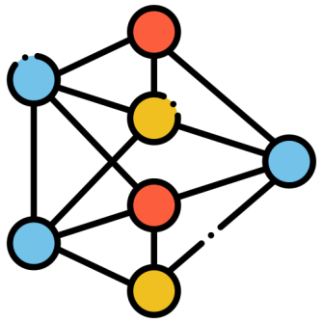
- Have lots of data
- Looking for model of data



# Evolutionary Computation + AI

- Surrogate fitness functions
- Hyper-parameter optimization
- NEAT algorithms



# Broader Perspective – Bio-Inspired Computing

| Swarm intelligence based algorithms | | | Bio-inspired (not SI-based) algorithms | | |
|---|---|---|---|---|---|
| Algorithm | Author | Reference | Algorithm | Author | Reference |
| Accelerated PSO | Yang et al. | [69], [71] | Atmosphere clouds model | Yan and Hao | [67] |
| Ant colony optimization | Dorigo | [15] | Biogeography-based optimization | Simon | [56] |
| Artificial bee colony | Karaboga and Basturk | [31] | Brain Storm Optimization | Shi | [55] |
| Bacterial foraging | Passino | [46] | Differential evolution | Storn and Price | [57] |
| Bacterial-GA Foraging | Chen et al. | [6] | Dolphin echolocation | Kaveh and Farhoudi | [33] |
| Bat algorithm | Yang | [78] | Japanese tree frogs calling | Hernández and Blum | [28] |
| Bee colony optimization | Teodorović and Dell'Orco | [62] | Eco-inspired evolutionary algorithm | Parpinelli and Lopes | [45] |
| Bee system | Lucic and Teodorovic | [40] | Egyptian Vulture | Sur et al. | [59] |
| BeeHive | Wedde et al. | [65] | Fish-school Search | Lima et al. | [14], [3] |
| Wolf search | Tang et al. | [61] | Flower pollination algorithm | Yang | [72], [76] |
| Bees algorithms | Pham et al. | [47] | Gene expression | Ferreira | [19] |
| Bees swarm optimization | Drias et al. | [16] | Great salmon run | Mozaffari | [43] |
| Bumblebees | Comellas and Martinez | [12] | Group search optimizer | He et al. | [26] |
| Cat swarm | Chu et al. | [7] | Human-Inspired Algorithm | Zhang et al. | [80] |
| Consultant-guided search | Iordache | [29] | Invasive weed optimization | Mehrabian and Lucas | [42] |
| Cuckoo search | Yang and Deb | [74] | Marriage in honey bees | Abbass | [1] |
| Eagle strategy | Yang and Deb | [75] | OptBees | Maia et al. | [41] |
| Fast bacterial swarming algorithm | Chu et al. | [8] | Paddy Field Algorithm | Premaratne et al. | [48] |
| Firefly algorithm | Yang | [70] | Roach infestation algorithm | Havens | [25] |
| Fish swarm/school | Li et al. | [39] | Queen-bee evolution | Jung | [30] |
| Good lattice swarm optimization | Su et al. | [58] | Shuffled frog leaping algorithm | Eusuff and Lansey | [18] |
| Glowworm swarm optimization | Krishnanand and Ghose | [37], [38] | Termite colony optimization | Hedayatzadeh et al. | [27] |
| Hierarchical swarm model | Chen et al. | [5] | Physics and Chemistry based algorithms | | |
| Krill Herd | Gandomi and Alavi | [22] | Big bang-big Crunch | Zandi et al. | [79] |
| Monkey search | Mucherino and Seref | [44] | Black hole | Hatamlou | [24] |
| Particle swarm algorithm | Kennedy and Eberhart | [35] | Central force optimization | Formato | [21] |
| Virtual ant algorithm | Yang | [77] | Charged system search | Kaveh and Talatahari | [34] |
| Virtual bees | Yang | [68] | Electro-magnetism optimization | Cuevas et al. | [13] |
| Weightless Swarm Algorithm | Ting et al. | [63] | Galaxy-based search algorithm | Shah-Hosseini | [53] |
| Other algorithms | | | Gravitational search | Rashedi et al. | [50] |
| Anarchic society optimization | Shayeghi and Dadashpour | [54] | Harmony search | Geem et al. | [23] |
| Artificial cooperative search | Civicioglu | [9] | Intelligent water drop | Shah-Hosseini | [52] |
| Backtracking optimization search | Civicioglu | [11] | River formation dynamics | Rabanal et al. | [49] |
| Differential search algorithm | Civicioglu | [10] | Self-propelled particles | Vicsek | [64] |
| Grammatical evolution | Ryan et al. | [51] | Simulated annealing | Kirkpatrick et al. | [36] |
| Imperialist competitive algorithm | Atashpaz-Gargari and Lucas | [2] | Stochastic difusion search | Bishop | [4] |
| League championship algorithm | Kashan | [32] | Spiral optimization | Tamura and Yasuda | [60] |
| Social emotional optimization | Xu et al. | [66] | Water cycle algorithm | Eskandar et al. | [17] |

Table 1. A list of algorithms

# Concluding Remarks

- Modern software problems require **innovative solutions**
- SBSE
  - View SWE problems as **search problems**
  - Use search-based optimization to **find solutions**
- EC
  - Nature is a **creative** problem solver
  - **EC** leverages creativity to solve SBSE problems

# Resources

- https://www.cs.vu.nl/~gusz/ecbook/ecbook-course.html
- https://www.cs.jhu.edu/~ayuille/courses/Stat202C-Spring10/ga_tutorial.pdf
- https://www.kaggle.com/code/zzettrkalpakbal/tutorial-of-genetic-algorithm
- https://github.com/kiecodes/genetic-algorithms